HDOC DAY 69

Recursive Fibonacci Series and Call Count

Aim Generate the first n Fibonacci numbers using recursion and count recursive function invocations.

Algorithm

1. Read n, the number of terms.

2. Initialize a global counter to zero.

3. Each invocation of fibonacci(k) increments the counter.

4. Use base cases fibonacci(0)=0 and fibonacci(1)=1.

5. Otherwise return fibonacci(k-1)+fibonacci(k-2).

6. Print fibonacci(0) through fibonacci(n-1), then display the total number of function calls.

C Program

```
#include <stdio.h>

unsigned long long calls = 0;

unsigned long long fibonacci(int n)
{
    calls++;
    if (n <= 1) return (unsigned long long)n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main(void)
{
    int n;
    printf("Enter number of terms: ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 35) {
        printf("Enter a term count from 1 to 35.\n");
        return 1;
    }
    printf("Fibonacci series: ");
    for (int i = 0; i < n; i++)
        printf("%llu ", fibonacci(i));
    printf("\nTotal recursive function calls: %llu\n", calls);
    return 0;
}
```

Test Cases

Input	Expected result
1	0; calls=1
5	0 1 1 2 3; calls=19
6	0 1 1 2 3 5; calls=34

Compilation and Execution (Mac Terminal)

```
clang -std=c99 -Wall -Wextra day69.c -o day69
./day69
```

Sample Output

```
Enter number of terms: 5
Fibonacci series: 0 1 1 2 3
Total recursive function calls: 19
```

Important Note

The counter includes every invocation of fibonacci(), including base-case calls and the initial calls made from main. This direct recursive method repeats calculations and is suitable for small n.

Result

The program was implemented using the required technique and produces the expected results for the given test cases.